

TWO-WAY DIGITAL PAGING SYSTEM

TEAM NETLINK

230058N - Aroshana H.A.P

230121D - De Mel D.J.

230539P - Ratheeshan A.R.

230680M - Wanigasundara W.M.H.



Communication Design Project

Electronic and Telecommunication Department
University of Moratuwa

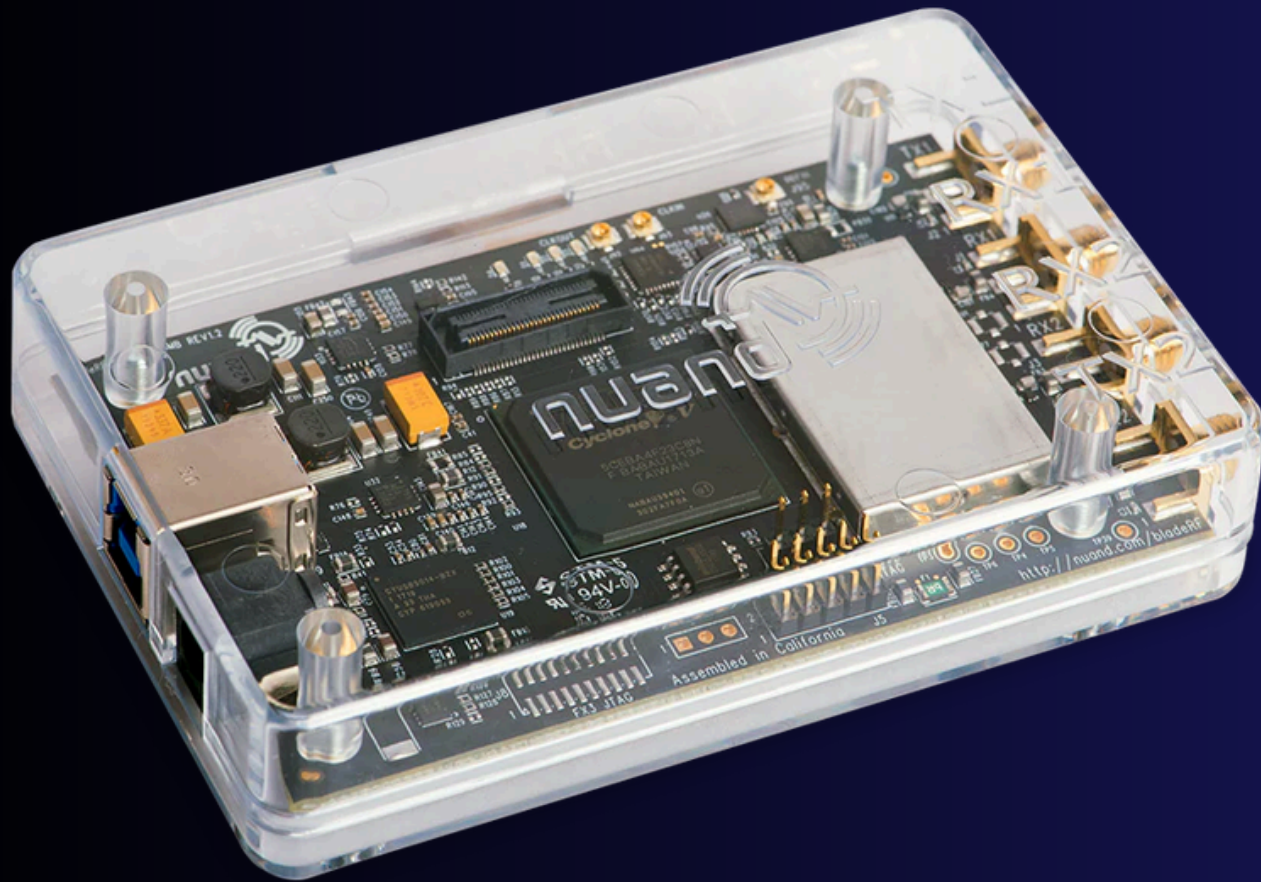
INTRODUCTION

Our project transforms standard radio into a Secure & Reliable Packet-Switched System capable of guaranteeing delivery in noisy environments. By building a custom protocol in GNU Radio, we integrated ARQ for reliability, AES encryption for security, and Priority Scheduling to ensure 'Code Red' alerts always cut through the traffic. This system provides dispatchers with a resilient, modern tool for critical communication.



Project Overview & Objectives

Our goal is to design a reliable two-way text messaging system using BladeRF and GNU Radio.



Core Requirements Met:

- Digital Modulation (QPSK via GNU Radio).
- Unique User Addressing (Src/Dest Logic).
- Reliability (ACK Mechanism).

Added Features Implemented:

- AES-128 Encryption for security.
- Priority-Based Handling (Emergency vs. Normal).
- Emergency Dashboard GUI (Python).

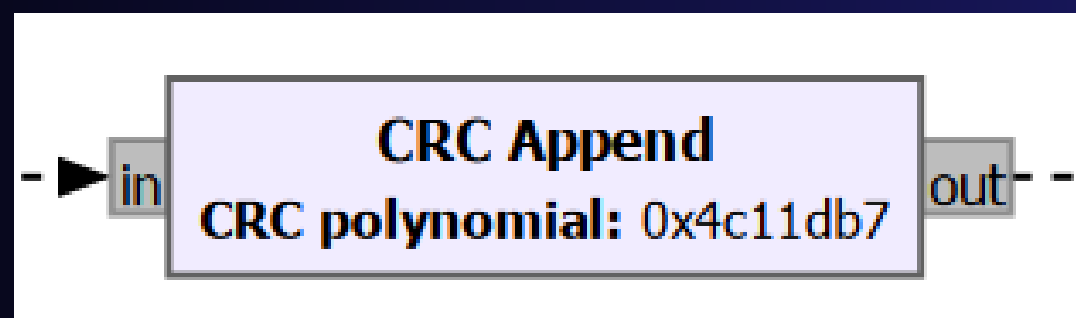
Use BladeRF for physical transmission.

Reliability & Acknowledgment

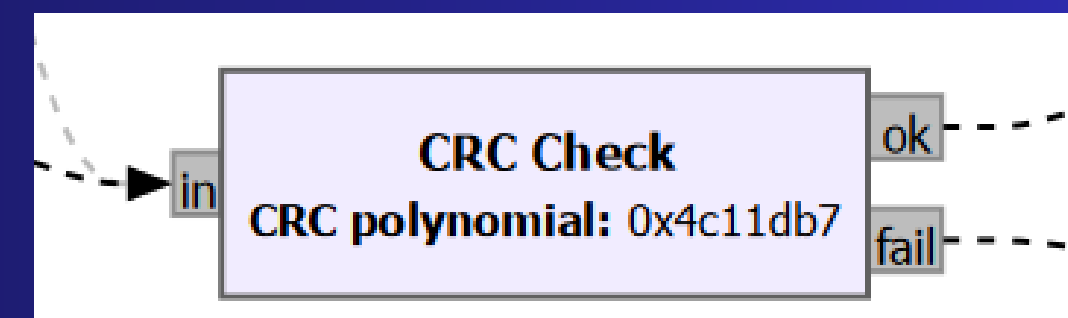
- Requirement: Acknowledgment mechanism (ACK) and Reliable data delivery.
- Our Solution: A custom ARQ (Automatic Repeat Request) Protocol.
- How it works:
 - a.Transmitter: Sends a packet and stores a copy in a "Waiting Buffer."
 - b.Receiver: Decodes the message and uses the ACK Generator block to swap IPs and bounce an 'ACK' packet back.
 - c.Feedback: If the transmitter receives the ACK, it clears the buffer. If not, it re-transmits automatically after a timeout.

Error Detection & Validation

- Requirement: Discard corrupted messages.
- Our Implementation: CRC implementation and Multi-stage Integrity Check.
 - a. Error detection : use CRC-32
 - b. Length Validation: Any packet less than 12 bytes is discarded immediately.
 - c. Header Logic: Invalid message types or nonsensical sequence numbers are rejected by the Sequence Decoder.

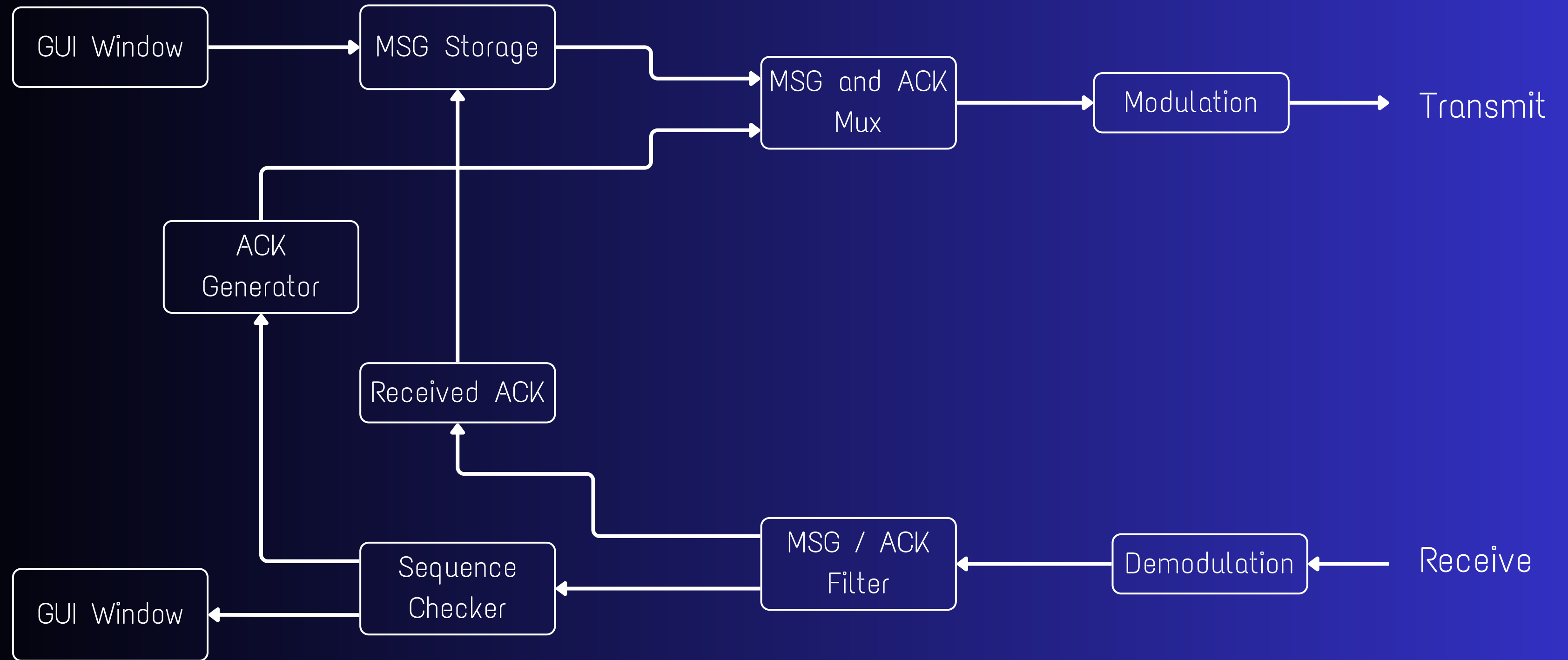


at Transmitter



at receiver

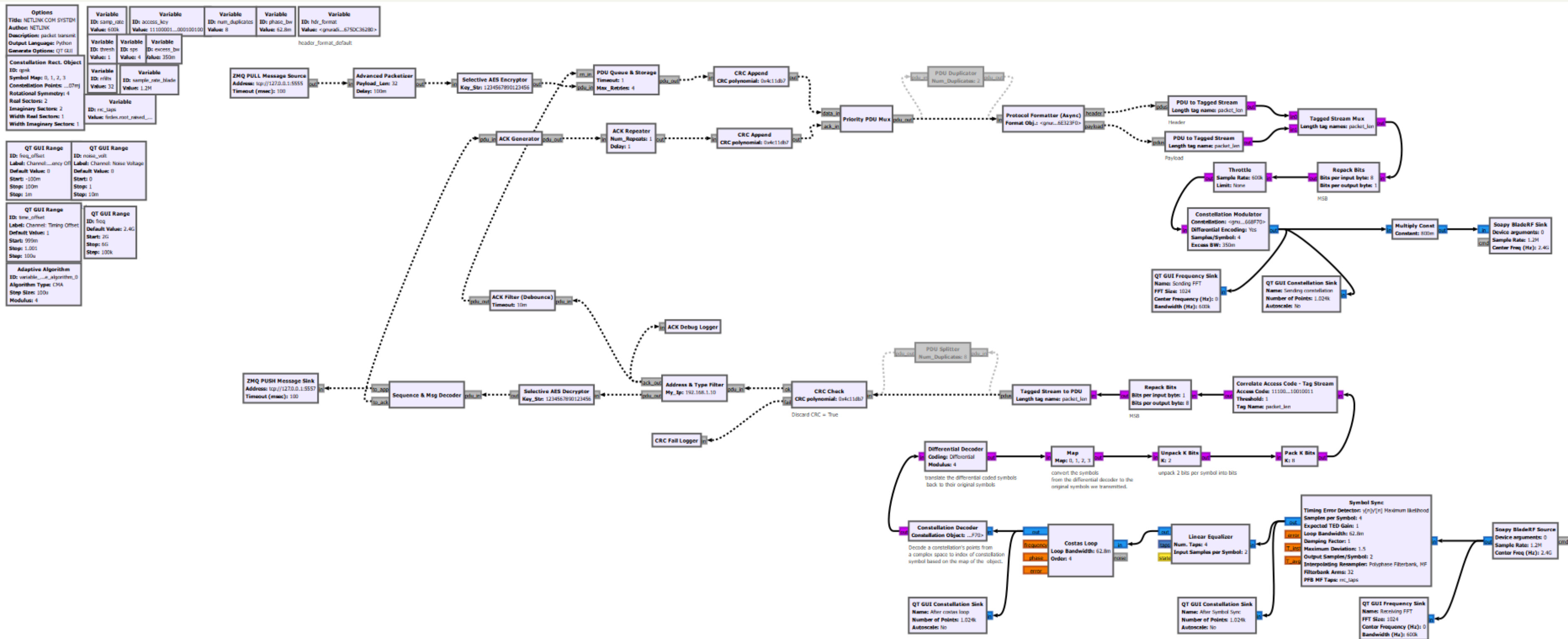
System Architecture



Flowgraph for one user

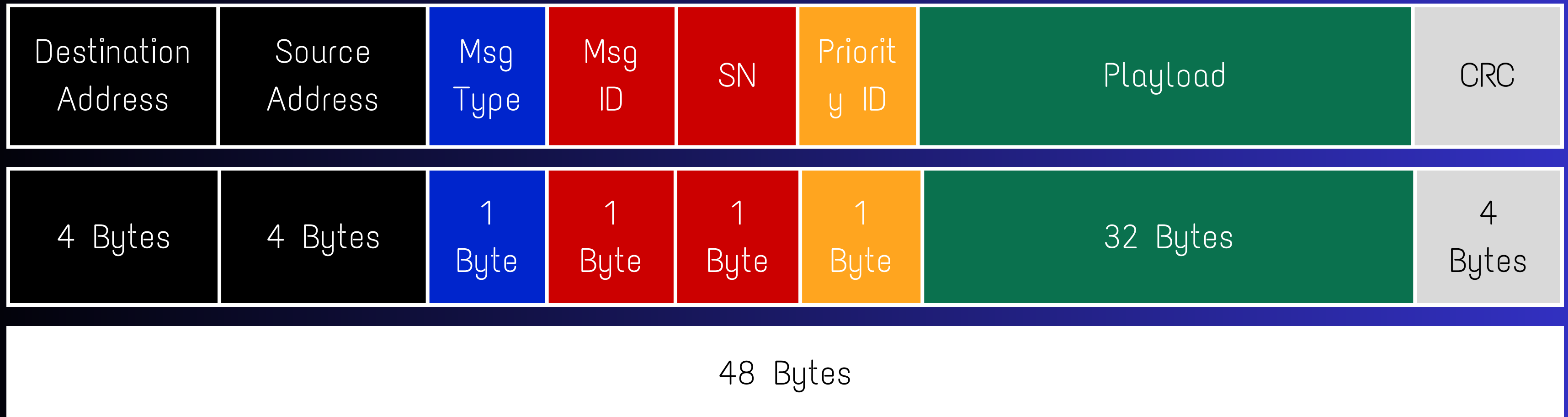
*Final_one_user_GNU.grc - C:\Users\arosh\OneDrive\Desktop\UOM resources\Sem 3\Com design\Project

File Edit View Run Tools Help



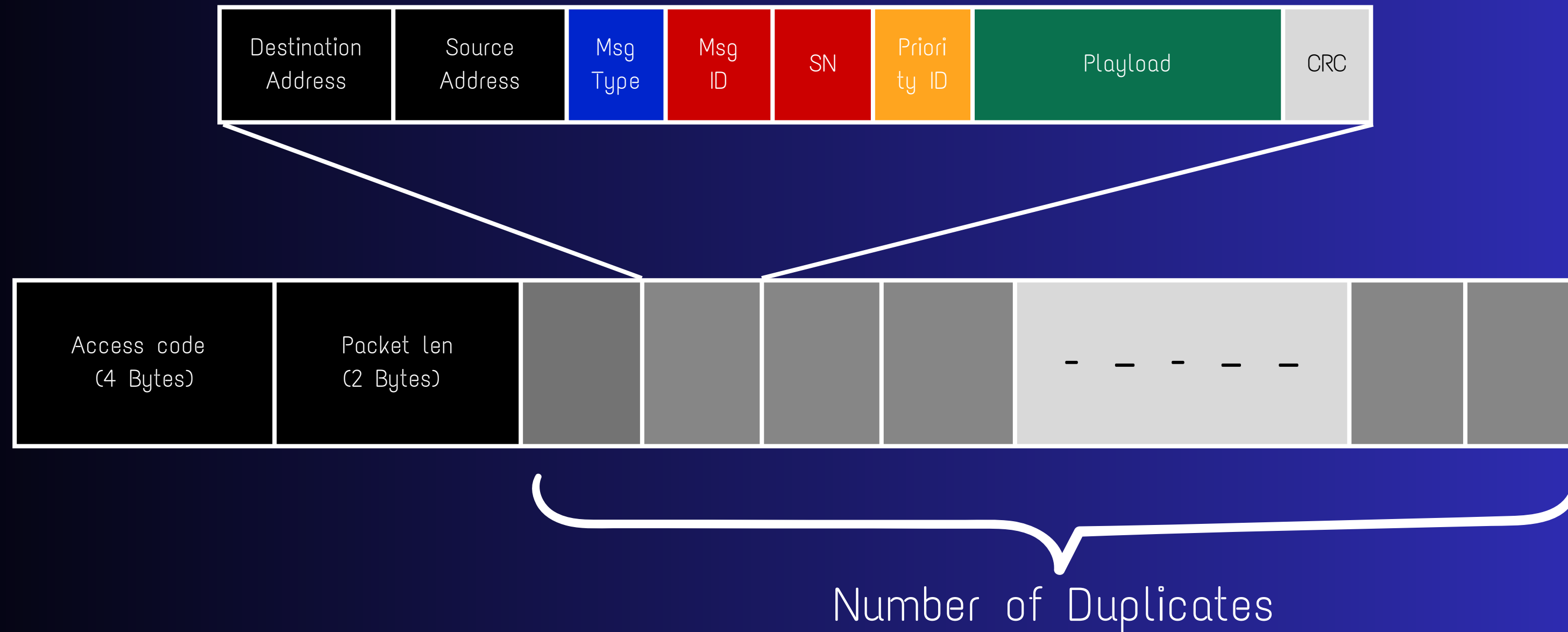
Frame Structure

The frame is structured as indicated bellow.



- Msg Type : specifies whether the packet is a message or an ack.
- Msg ID : keep track of the message number being sent.
- SN (Sequence number) : sequence number for the message of the frame sent.
- Priority ID : Priority level containing E (Emergency) or N (Normal).

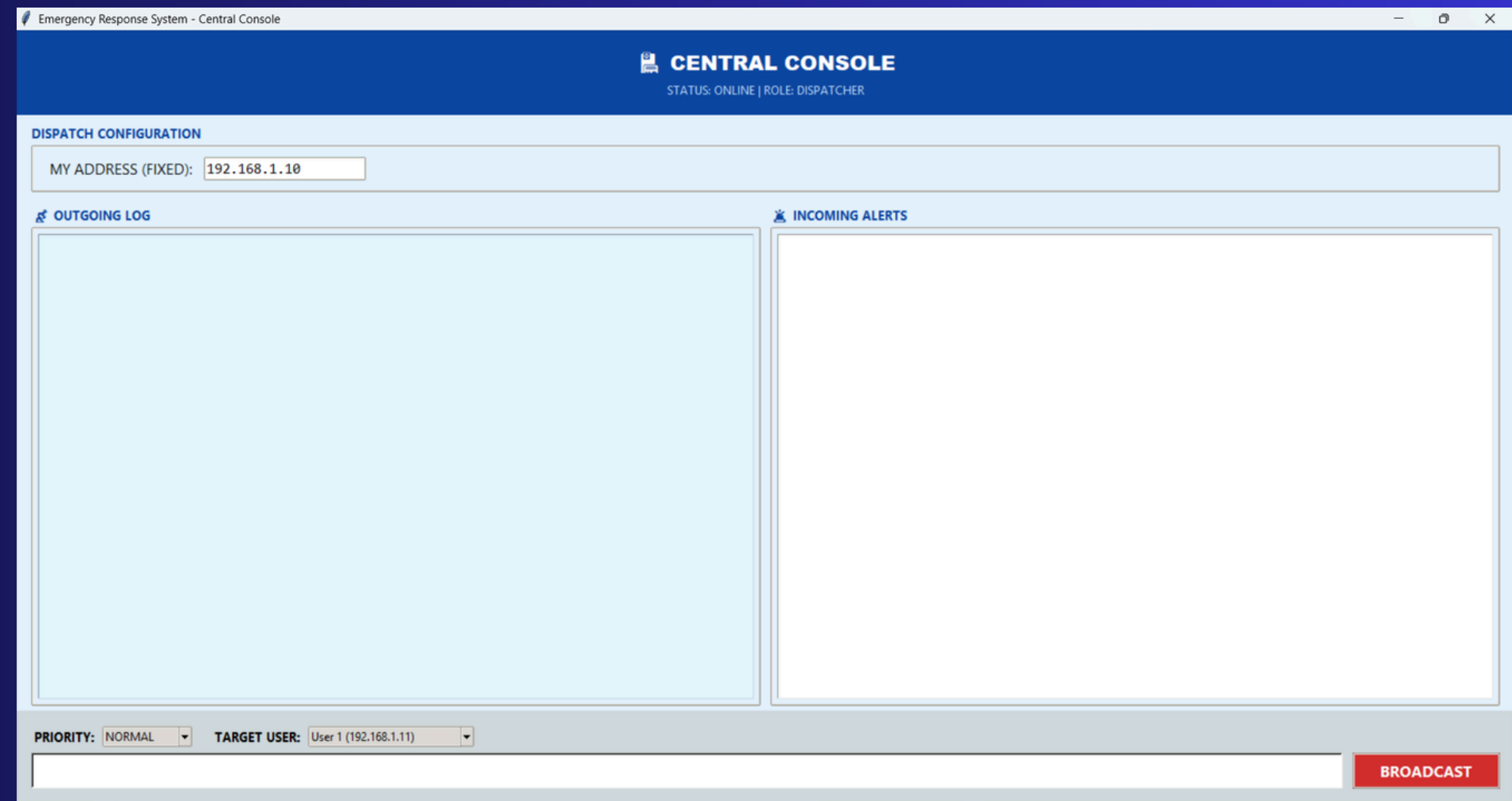
Frame Structure



- We are duplicating packets to Sync Stabilization and increase Resistance to Burst Noise

GUI (Hospital Paging System)

- Built with Python Tkinter.
- Split-Log View: Separates "Outgoing" and "Incoming" traffic for clarity.
- Real-Time: Uses Threading to receive messages without freezing the interface.
- Functionality: Users can toggle "Priority" and set "Target Unit IDs" dynamically.



Graphical User Interface (GUI)

The screenshot shows a web application window titled "Emergency Response System - User Dashboard". The main header is blue with a white truck icon and the text "USER DASHBOARD" and "STATUS: ONLINE". Below the header, there are several sections:

- DEVICE CONFIGURATION (WHO AM I?)**: A light blue box containing two input fields: "SELECT MY IDENTITY:" with a dropdown menu showing "User 1", and "MY IP ADDRESS:" with a text input field containing "192.168.1.11".
- OUTGOING LOG**: A light blue box with a list of outgoing messages.
- INCOMING ALERTS**: A light blue box with a list of incoming alerts.
- PRIORITY:** A dropdown menu showing "NORMAL".
- TARGET DESTINATION:** A dropdown menu showing "User 3 (192.168.1.13)".
- Message Typing**: A text input field for entering the message content.
- BROADCAST**: A red button to send the message.

Annotations with callout boxes point to specific parts of the interface:

- Sender**: Points to the "SELECT MY IDENTITY:" dropdown menu.
- Sending Messages**: Points to the "OUTGOING LOG" section.
- Emergency / Normal**: Points to the "PRIORITY:" dropdown menu.
- Message Typing**: Points to the text input field for entering the message content.
- Receiving Message**: Points to the "INCOMING ALERTS" section.
- Destination Address**: Points to the "TARGET DESTINATION:" dropdown menu.

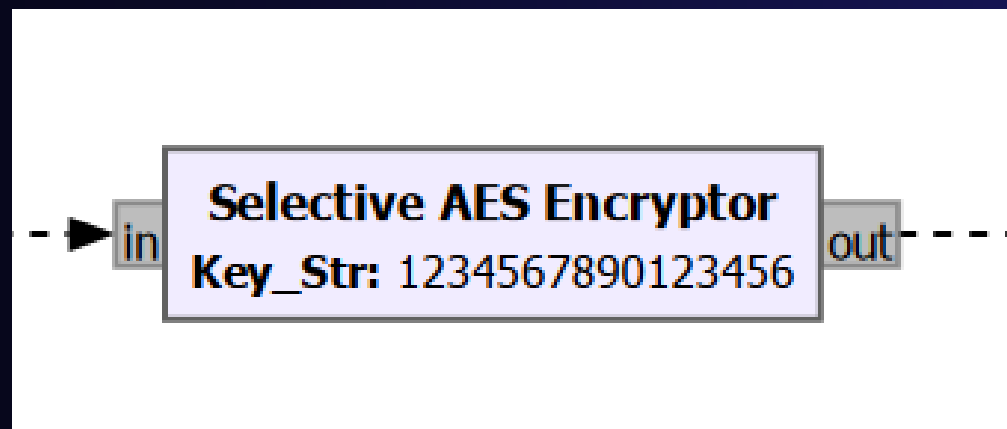
AES Encryption

Feature: "AES encryption for messages".

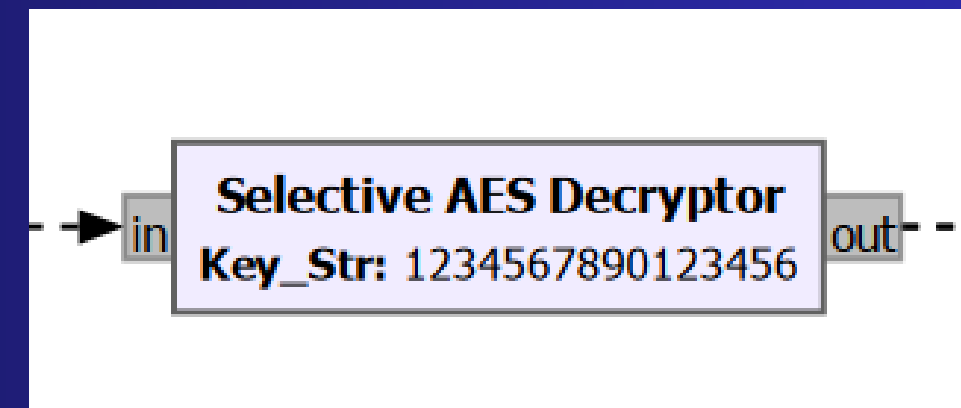
Implementation:

- Selective Encryption: The Header remains cleartext (for routing), but the Payload is encrypted using AES-128 (ECB Mode).

Result: Even if a signal is intercepted, the text content is unreadable without the correct key.



Encryption at transmitter



Decryption at receiver

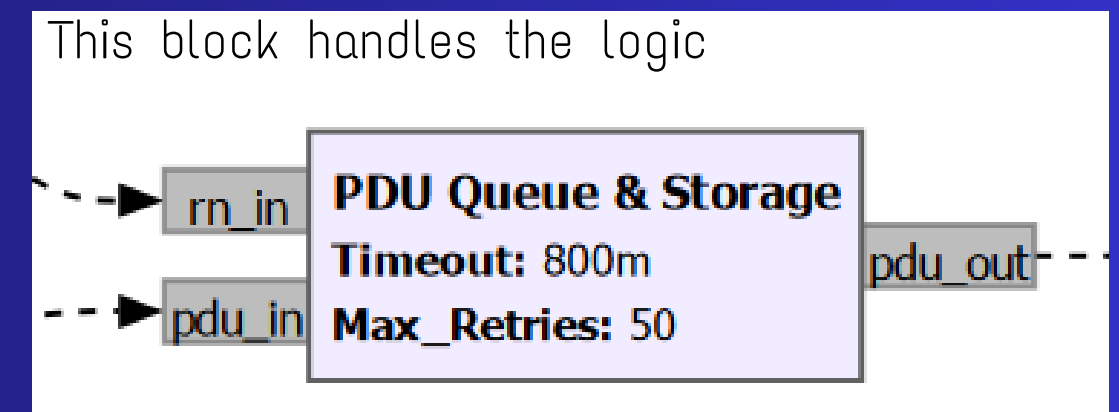
Priority-based message handling

Feature : "Priority-based message handling".

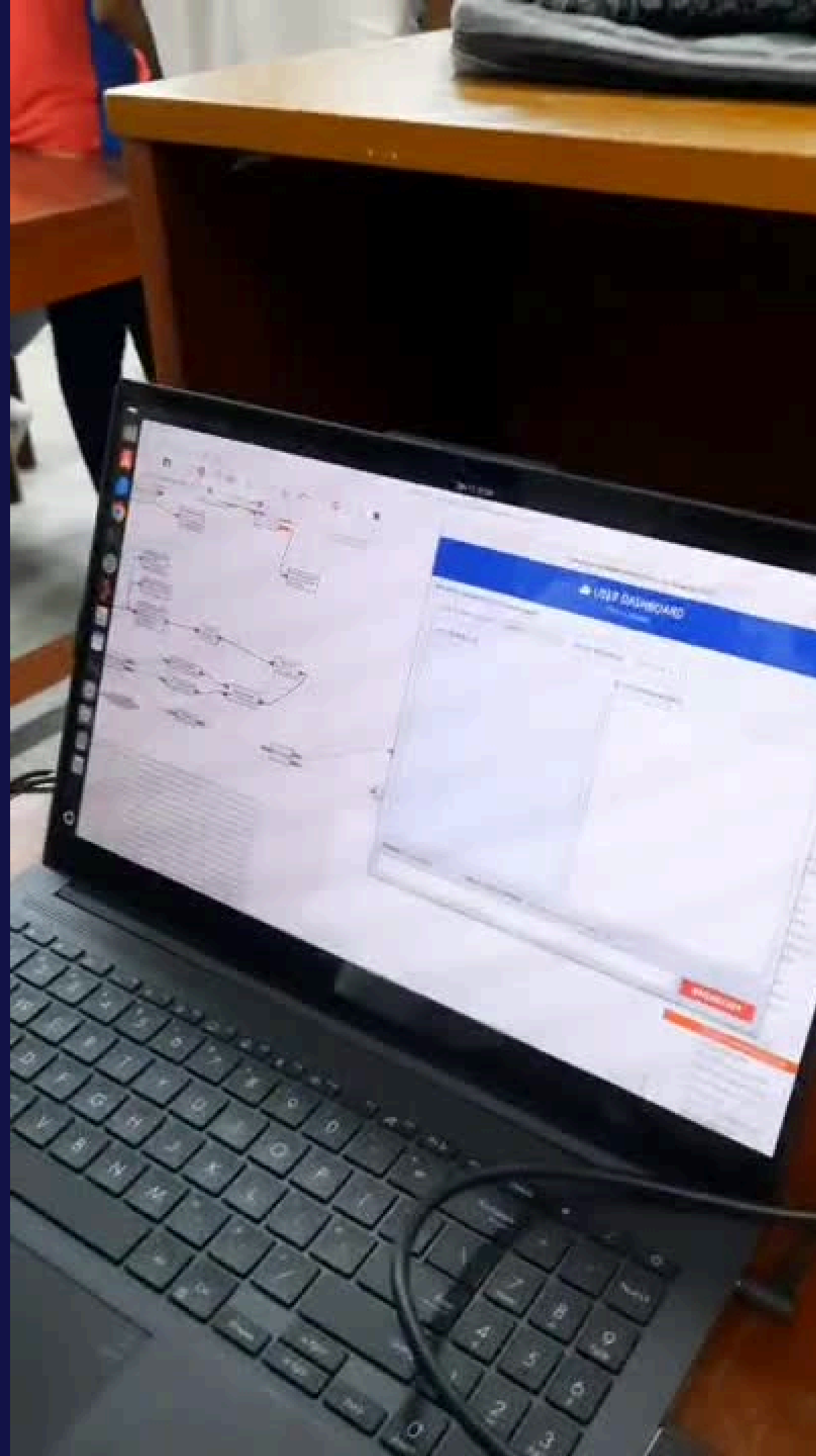
Implementation:

- Developed a 5-Level Priority Queue in the transmitter.
 1. New Emergency Packets
 2. Emergency Retransmissions
 3. Normal Retransmissions
 4. New Normal Packets
 5. Persistent Storage (Background)
- Logic: An incoming "Emergency" message immediately interrupts any "Normal" messages currently in the buffer.

Visual : Emergency messages appear in Bold Red on the dashboard,
Normal messages in Blue.



Testing



Thank You.